

Exercices — Le Tas (Heap)

AP2 — ENSIMAG 1ère année

Rappel : Qu'est-ce qu'un Tas ?

Un **tas** est un arbre binaire complet stocké dans un tableau. Pour un noeud à l'indice i :

- Fils gauche : indice $2i + 1$
- Fils droit : indice $2i + 2$
- Père : indice $(i - 1) // 2$

Propriété du max-tas : chaque noeud est supérieur ou égal à ses enfants. **Propriété du min-tas** : chaque noeud est inférieur ou égal à ses enfants.

Exercice 1 — Représentation tableau/arbre

Soit le tableau suivant représentant un tas :

```
[50, 30, 40, 10, 20, 35, 25]
```

1. Dessinez l'arbre binaire correspondant à ce tableau.
2. Vérifiez que c'est bien un max-tas.
3. Quels sont les indices des feuilles ?
4. Quel est le père du noeud à l'indice 5 ?

Exercice 2 — Vérification d'un tas

Pour chacun des tableaux suivants, dites s'il représente un max-tas, un min-tas, ou ni l'un ni l'autre :

- a) [100, 50, 80, 20, 30, 60, 70]
- b) [1, 3, 2, 7, 5, 4, 6]
- c) [10, 20, 15, 30, 25]
- d) [5, 5, 5, 5, 5]

Exercice 3 — Insertion dans un tas

L'insertion dans un max-tas fonctionne ainsi : on ajoute l'élément à la fin du tableau, puis on le fait **remonter** (percolate up) tant qu'il est plus grand que son père.

Soit le max-tas : [90, 70, 80, 40, 60, 50, 30]

1. Insérez la valeur **85**. Montrez chaque étape.
2. Insérez ensuite la valeur **20**. Montrez chaque étape.
3. Quel est le coût (en nombre de comparaisons) de l'insertion ? Justifiez.

Exercice 4 — Suppression du maximum

La suppression du maximum dans un max-tas fonctionne ainsi : on remplace la racine par le dernier élément du tableau, on retire ce dernier élément, puis on fait **descendre** la racine (percolate down) tant qu'elle est plus petite qu'un de ses enfants.

Soit le max-tas : [100, 60, 80, 30, 50, 70, 40]

1. Supprimez le maximum. Montrez chaque étape.
2. Supprimez à nouveau le maximum. Montrez chaque étape.
3. Quel est le coût de la suppression ? Justifiez.

Exercice 5 — Construction d'un tas (Heapify)

Soit le tableau non ordonné suivant :

[3, 10, 5, 1, 8, 6, 9, 2, 7, 4]

L'algorithme **heapify** construit un max-tas en appliquant percolate down sur tous les noeuds non-feuilles, de droite à gauche.

1. Identifiez les indices des noeuds non-feuilles.
2. Appliquez heapify et montrez le tableau final.
3. Quel est le coût de heapify ? ($O(n)$ ou $O(n \log n)$?) Justifiez.

Exercice 6 — Tri par tas (Heap Sort)

Le **tri par tas** fonctionne en deux phases :

- Phase 1 : construire un max-tas avec heapify.
- Phase 2 : extraire le maximum n fois et le placer à la fin.

Triez le tableau suivant avec heap sort :

[4, 2, 7, 1, 9, 3]

1. Montrez le tableau après la phase heapify.
2. Montrez chaque étape de la phase d'extraction.

3. Quel est le coût total du tri par tas en fonction de n ?

Exercice 7 — Implémentation en Python

Implémentez les fonctions suivantes pour un **max-tas** représenté par une liste Python :

1. `pere(i)` — retourne l'indice du père de i .
2. `fils_gauche(i)` — retourne l'indice du fils gauche.
3. `fils_droit(i)` — retourne l'indice du fils droit.
4. `est_tas(tab)` — retourne `True` si `tab` est un max-tas.
5. `insérer(tab, val)` — insère `val` dans le max-tas.
6. `supprimer_max(tab)` — supprime et retourne le maximum.

Résumé des Complexités

Opération	Coût
Accès au maximum	$O(1)$
Insertion	$O(\log n)$
Suppression du max	$O(\log n)$
Construction (heapify)	$O(n)$
Tri par tas	$O(n \log n)$